NIT2112
Object Oriented Programming

Session 04

Abstraction and Interfaces

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Acknowledgement of Country

Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

Session Outline

- Abstraction: Hiding Complexity (Concept & Usage)
- Interfaces: Defining Contracts (Concept & Usage)
- Abstraction vs. Interfaces: Differences & Benefits
- Best Practices for Maintainable Code

Abstraction: Hiding Complexity

- A fundamental OOP principle.
- Focuses on essential characteristics.
- Hides irrelevant implementation details.
- Simplifies complex systems.

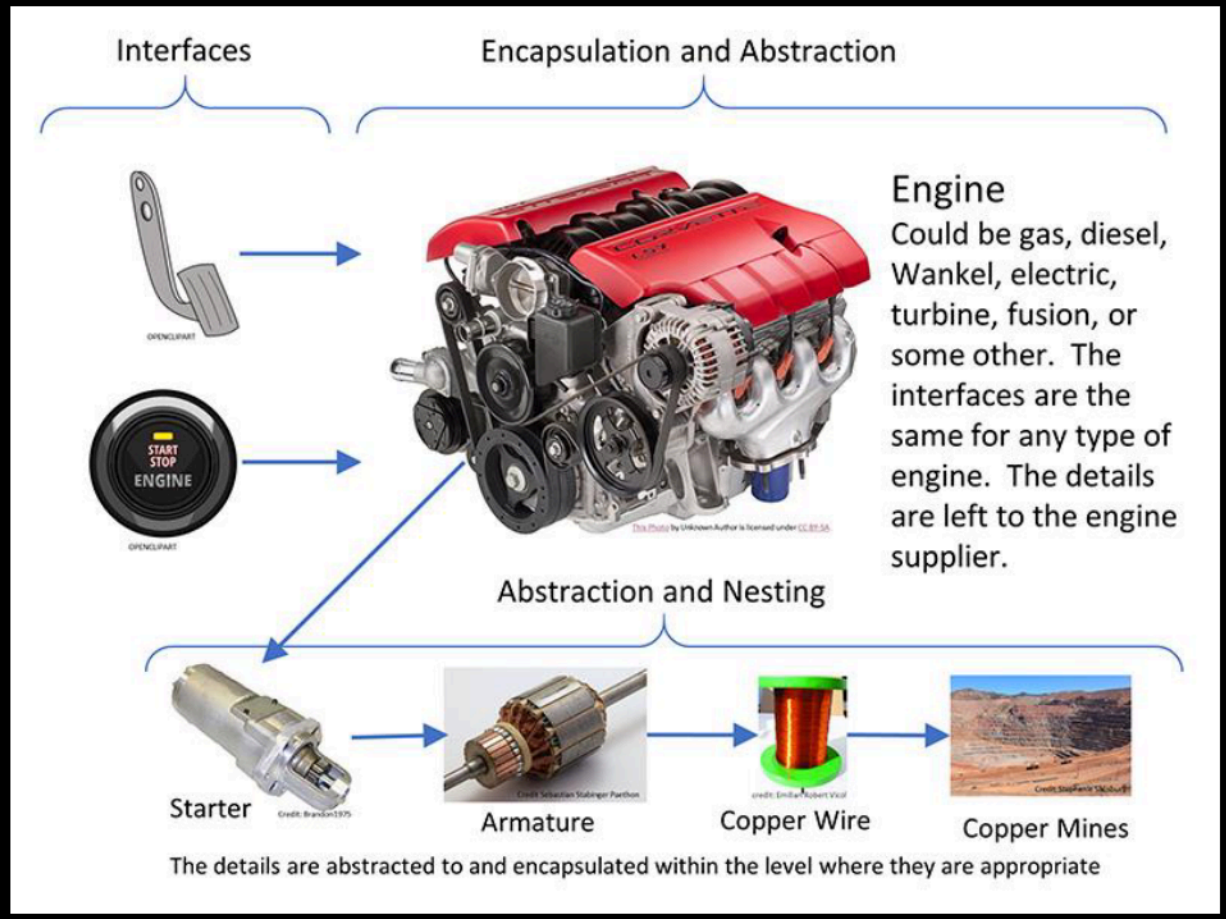


Image from Gary L. Pratt, P.E., October 01, 2020, https://www.automation.com/en-us/articles/october-2020/oojp-part-2-abstraction-nesting-and-interfaces

Abstraction in the Real World: Coffee Machine

- You interact with the coffee machine by pressing buttons (e.g., "Brew", "Steam").
- You don't need to know how the machine heats the water, grinds the beans, or steams.
- The internal mechanisms are hidden (abstracted) away.

Image from Freerange Stock, 2025,
https://freerangestock.com/photos/167091/espresso-being-poured-into-a-cup.html

Abstraction in OOP: Classes as Blueprints

- **Classes** are **blueprints** for creating **objects**.
- They define **attributes** (data) and **methods** (behaviour).
- Abstraction defines **what** an object does, not **how** it does it.

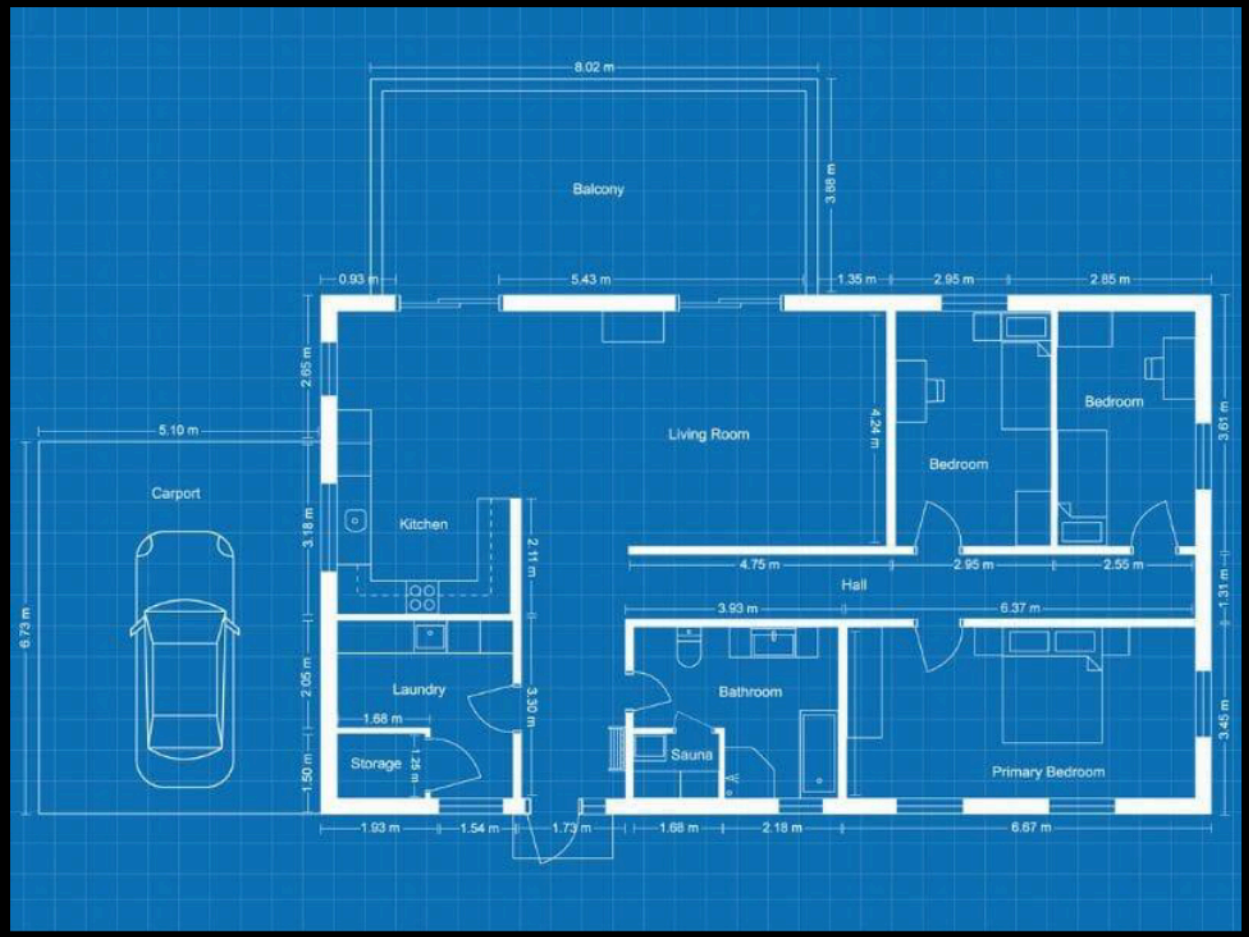


Image from RoomSketcher AS, 2025, https://www.roomsketcher.com/floor-plans/blueprint-maker

Specialization through Inheritance

- **Inheritance** allows creating **specialized** classes (subclasses) from a general class (superclass).
- Subclasses inherit **attributes** and **methods** from the superclass.
- Subclasses can **override** methods to provide specific implementations.

```python
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def make_sound(self):
        print("Generic animal sound")

    def move(self):
        print("Generic animal movement")
```

```python
class Dog(Animal):
    def __init__(self, name):
        super().__init__(name, "Dog")

    def make_sound(self):
        print("Woof!")

    def move(self):
        print("Run fast!")

class Cat(Animal):
    def __init__(self, name):
        super().__init__(name, "Cat")

    def make_sound(self):
        print("Meow!")

    def move(self):
        print("Walk smoothly")
```

Abstract Classes and Methods: Defining a Blueprint

- An abstract class **cannot** be instantiated directly.
- It serves as a blueprint for other classes.
- Abstract methods have **no implementation** in the abstract class; subclasses **must** provide the implementation.
- Used to **enforce** a certain structure and behaviour in subclasses.

Using Abstract Base Classes (abc Module)

- Python provides the *abc* module to define abstract classes.
- Use **abstract base classes (ABCs)** as the base class for abstract classes.
- Use the *@abstractmethod* decorator to define abstract methods.
- If the subclass does not implement all abstract methods, a *TypeError* will be raised when trying to instantiate the subclass.

```python
from abc import ABC, abstractmethod

# Abstract Class: AbstractAnimal
# (provides a base implementation + requires some specific behavior)
class AbstractAnimal(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def make_sound(self):  # Concrete sub classes MUST implment this method.
        """Make sound"""
        pass

    def describe(self): # Provides a default implmention
        return f"This animal is named {self.name}."

# Concrete Class Inheriting from AbstractAnimal
class Dog(AbstractAnimal):
    def __init__(self, name):
        super().__init__(name)

    def make_sound(self):
        return "Woof!"

# Create Objects
my_dog = Dog("Buddy")

# Use the Methods
print(my_dog.describe()) # Output: This animal is named Buddy.
print(my_dog.make_sound())  # Output: Woof!
```

Advantages of Abstraction

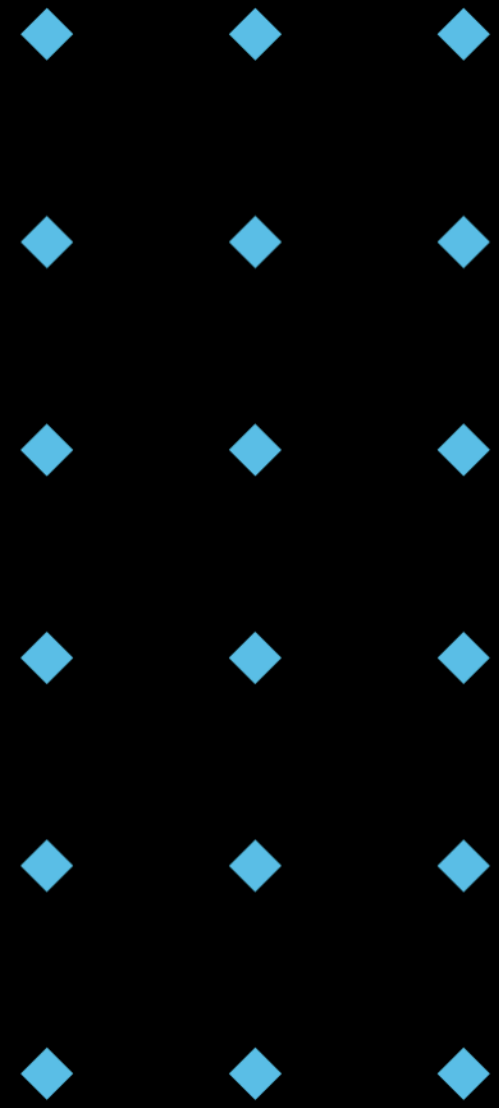
- **Reduced Complexity:** Makes code easier to understand.
- **Increased Reusability:** Abstract components can be reused in different parts of the program.
- **Improved Maintainability:** Changes to the implementation don't affect the overall structure.
- **Enhanced Modularity:** Components can be developed and tested independently.

Challenges of Abstraction

- **Design Complexity:** Finding the right level of abstraction can be difficult.
- **Performance Overhead:** Too much abstraction can lead to performance issues.
- **Understanding & Documentation:** Clear documentation is crucial for others to understand the abstraction.
- **Over-Abstraction:** Abstraction could be overused in simple cases.

Discussion Time!

- Question: Can you think of another real-world example of abstraction?



Interfaces: Defining Contracts

- An **interface** defines a **contract** for **classes**.
- Specifies what **methods** a class must implement.
- **Does not** specify **how** those methods are implemented.
- Enables **polymorphism** and **loose coupling**.

Image from Kaboompics, May 10, 2021, https://www.pexels.com/photo/business-people-discussing-details-in-the-contract-7875986/

Interfaces in Python: Abstract Base Classes

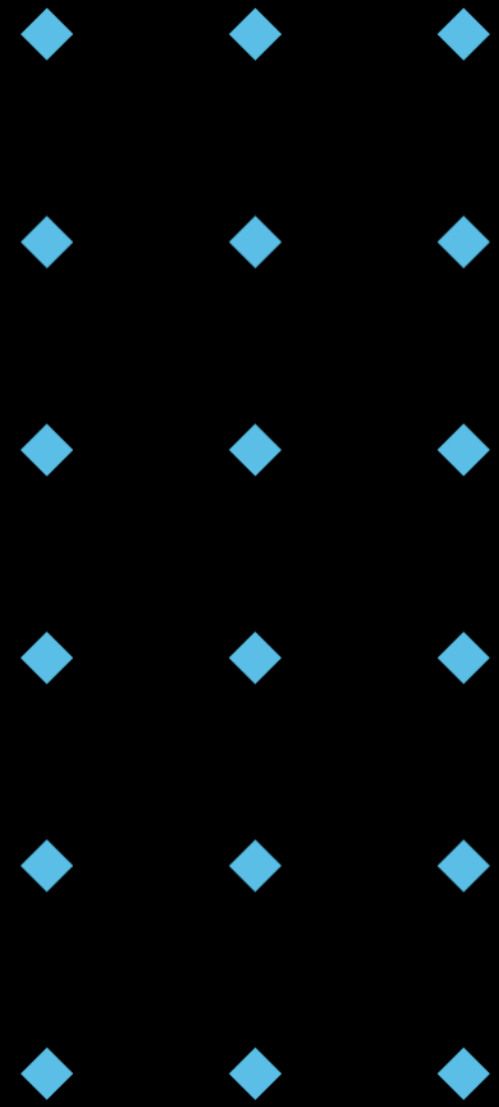
- Python doesn't have explicit "interface" keyword like Java.
- We use **abstract base classes (ABCs)** from the *abc* module to define interfaces.
- *ABCs* enforce that **subclasses** implement specific methods.

```python
from abc import ABC, abstractmethod

# Interface: ISpeak
class ISpeak(ABC):
    @abstractmethod
    def speak(self):
        """Returns the sound the object makes."""
        pass

```

```python
# Concrete Class Implementing ISpeak
class Parrot(ISpeak):
    def __init__(self, phrase):
        self.phrase = phrase

    def speak(self):
        return self.phrase

# Concrete Class Implementing ISpeak
class Speaker(ISpeak):
    def __init__(self, word):
        self.word = word

    def speak(self):
        return self.word

# Create Objects
my_parrot = Parrot("Hello!")
my_speaker = Speaker("Lecture!")

def make_them_speak(speaker: ISpeak): # type hint here is great!
    print(speaker.speak())

make_them_speak(my_parrot)  # Output: Hello!
make_them_speak(my_speaker)  # Output: Lecture!
```

Advantages of Using Interfaces

- **Loose Coupling:** Reduces dependencies between classes.
- **Polymorphism:** Allows objects of different classes to be treated uniformly.
- **Testability:** Easier to mock and test components independently.
- **Flexibility:** Enables interchangeable components.



Think-Pair-Share

- Question: How do interfaces improve code maintainability?

* Take a minute to think individually, then discuss with a partner, and finally share your thoughts with the class.

Abstract Classes vs. Interfaces: Method Definition

- Abstract Classes: Can have both **abstract** and **concrete** methods.
- Interfaces (in Python using ABCs): Typically only have **abstract** methods (though concrete methods are technically allowed).

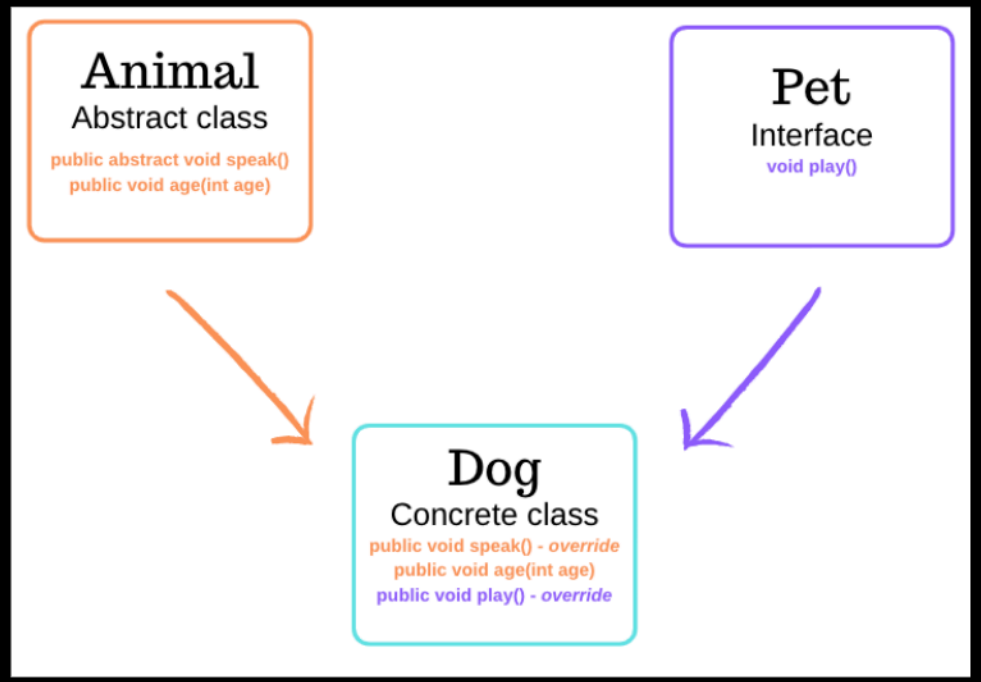
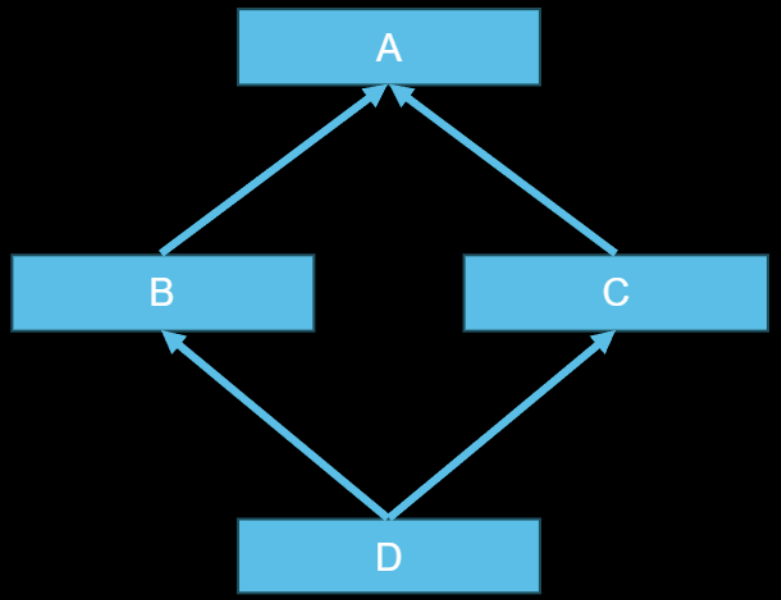


Image from Elle Hallal, May 29, 2019, https://www.ellehallal.dev/blog/2019/05/2019-05-29-what-are-interfaces-abstract-classes-and-concrete-classes/

Abstract Classes vs. Interfaces: Multiple Inheritance

- Interfaces: Python allows **multiple inheritance** of interfaces.
- Abstract Classes: Multiple inheritance can lead to the **"diamond problem"** and is generally discouraged for abstract classes with state or concrete methods.




```python
class Animal:
    def act(self):
        print("General Action")

class Swimmer(Animal):
    def act(self):
        print("Swimming")

class Flyer(Animal):
    def act(self):
        print("Flying")

class FlyingFish(Swimmer, Flyer):
    pass

fish = FlyingFish()
fish.act()  # What is the output?
```

Abstract Classes vs. Interfaces: State

- Abstract Classes: Can maintain **state** (instance variables).
- Interfaces: Typically do not maintain state. They define **behaviour**, not data.

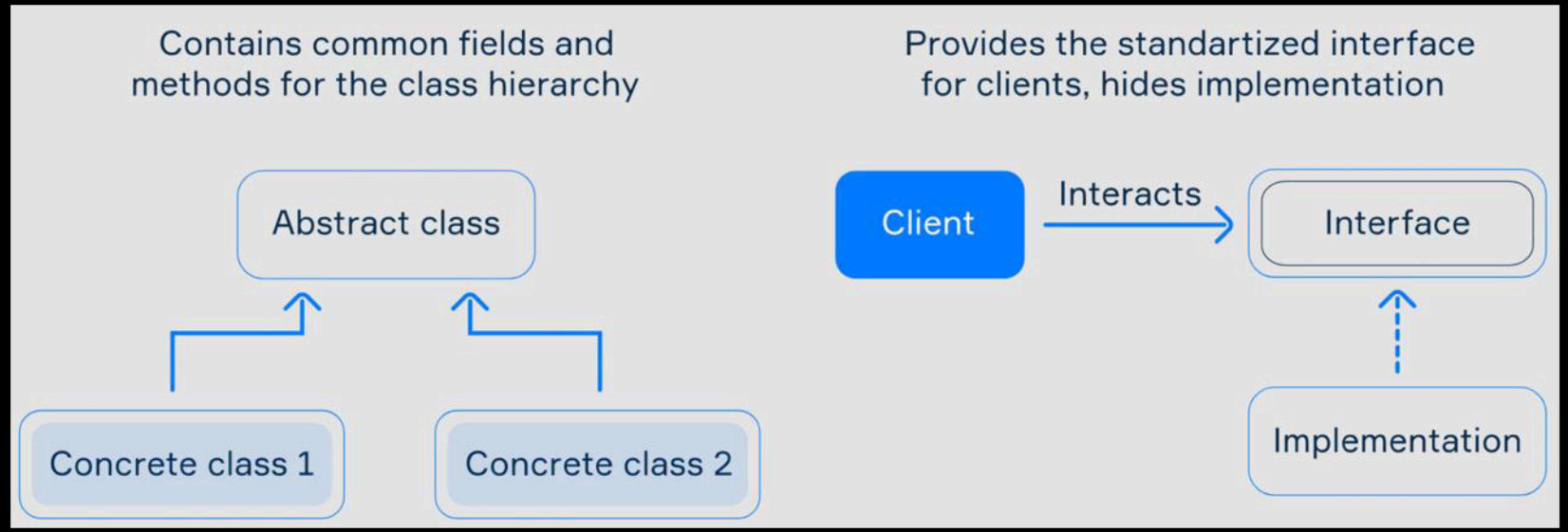


Image from Hyperskill, 2025, https://hyperskill.org/learn/step/3563

When to Use Abstract Classes & Interfaces

Abstract Classes	Interfaces
When you want to provide a common base implementation for a set of **related** classes.	When you want to define a contract that **unrelated** classes can implement.
When you want to share code and state among **subclasses**.	When you want to achieve polymorphism **without imposing a class hierarchy**.
When you want to **enforce a specific structure** for subclasses, but also provide some default behaviour.	When you want to **decouple** components and make them easily replaceable.

Example: Payment Processing System

```python
from abc import ABC, abstractmethod

class PaymentProcessor(ABC):
    @abstractmethod
    def authorize_payment(self, amount):
        pass

    @abstractmethod
    def execute_payment(self, amount):
        pass

class CreditCardProcessor(PaymentProcessor):
    def authorize_payment(self, amount):
        print(f"Authorizing credit card payment of ${amount}")
        # Implement credit card authorization logic

    def execute_payment(self, amount):
        print(f"Executing credit card payment of ${amount}")
        # Implement credit card payment execution logic

class PayPalProcessor(PaymentProcessor):
    def authorize_payment(self, amount):
        print(f"Authorizing PayPal payment of ${amount}")
        # Implement PayPal authorization logic

    def execute_payment(self, amount):
        print(f"Executing PayPal payment of ${amount}")
        # Implement PayPal payment execution logic
```

```python
def process_order(payment_processor, amount):
    payment_processor.authorize_payment(amount)
    payment_processor.execute_payment(amount)

credit_card_processor = CreditCardProcessor()
paypal_processor = PayPalProcessor()

process_order(credit_card_processor, 100)  # Process with credit card
process_order(paypal_processor, 50)      # Process with PayPal
```

Poll Question

- True or False: An abstract class can have concrete (implemented) methods.

Python's Flexibility: Duck Typing

- "If it walks like a duck and quacks like a duck, then it must be a duck."
- Python is **dynamically typed**, so we often don't need explicit interfaces.
- If an object has the methods we need, we can use it, regardless of its type.

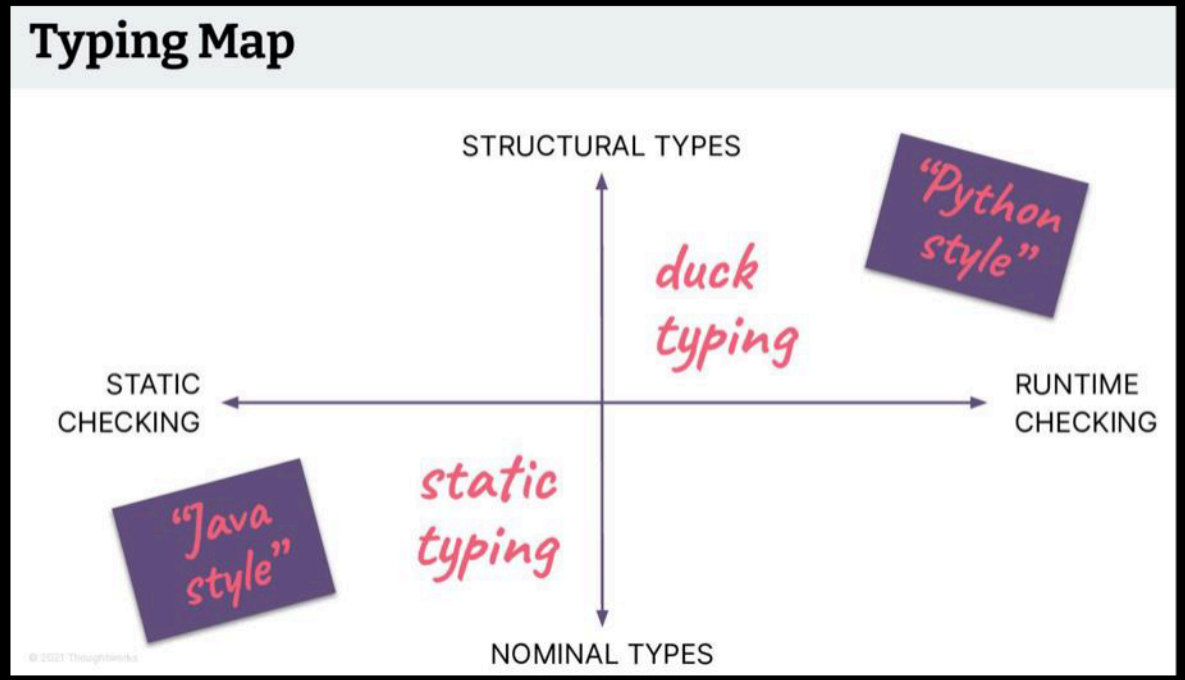


Image from Luciano Ramalho, July 15, 2022, https://speakerdeck.com/ramalho/typing-dot-protocol-type-hints-as-guido-intented

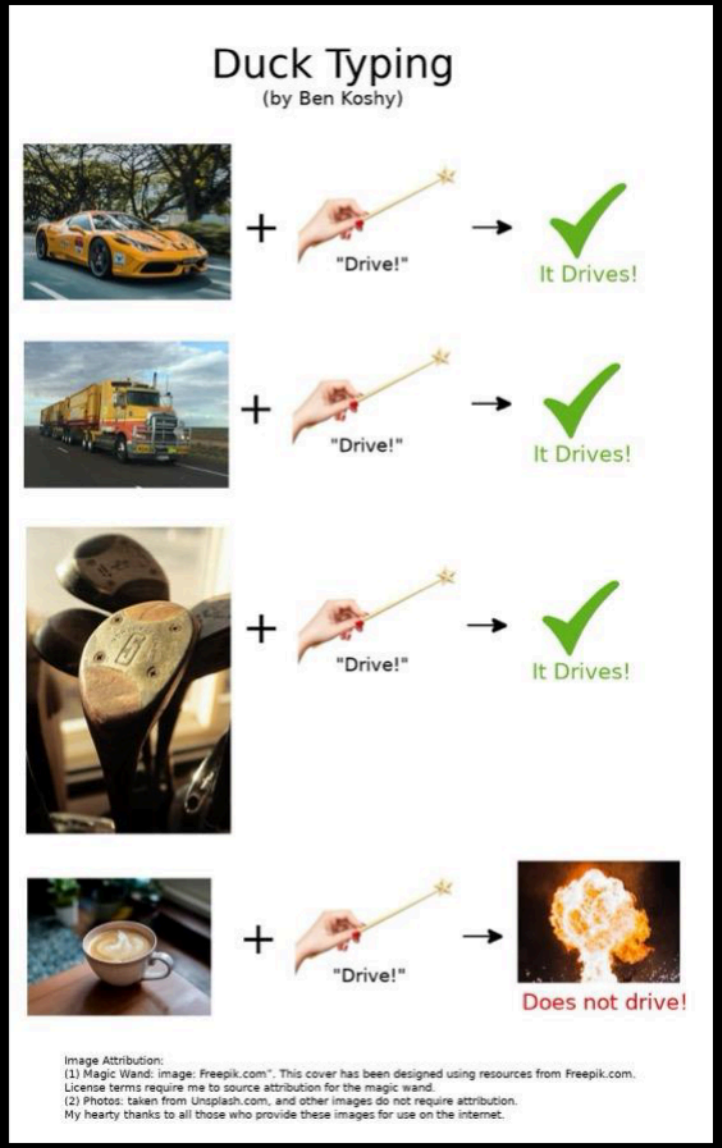


Image from BenKoshy, Nov 5, 2016, https://stackoverflow.com/a/40434829

Duck Typing Example

```python
class EmailSender:
    def send(self, message):
        print(f"Sending email: {message}")

class SMSSender:
    def send(self, message):
        print(f"Sending SMS: {message}")

def send_notification(sender, message):
    sender.send(message)

email_sender = EmailSender()
sms_sender = SMSSender()

send_notification(email_sender, "Hello via email!")
send_notification(sms_sender, "Hello via SMS!")
```

Best Practices: Clear Abstractions

- Focus on **essential features**, hiding implementation details.
- Use **meaningful** names for classes, methods, and variables.
- **Document** your abstractions clearly.

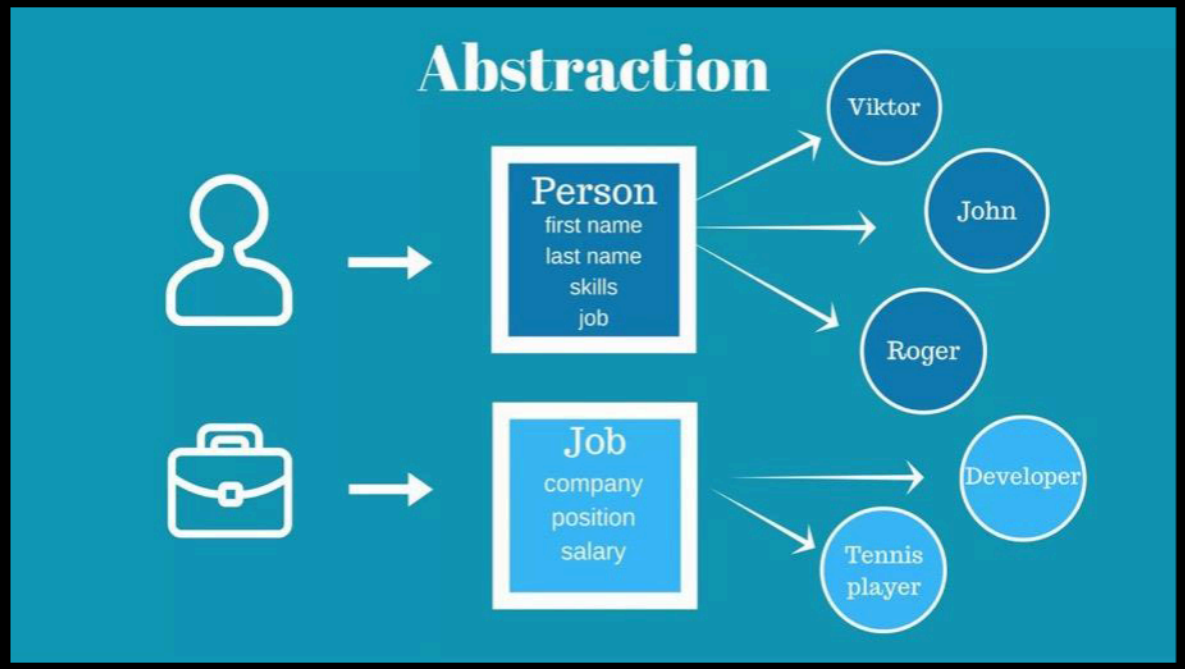


Image from Logicmojo, Dec 12, 2023, https://logicmojo.com/abstraction-in-oops

Best Practices: Single Responsibility Principle

- Each class and interface should have **one**, and **only one**, reason to change.
- Avoid "god classes" that do everything.

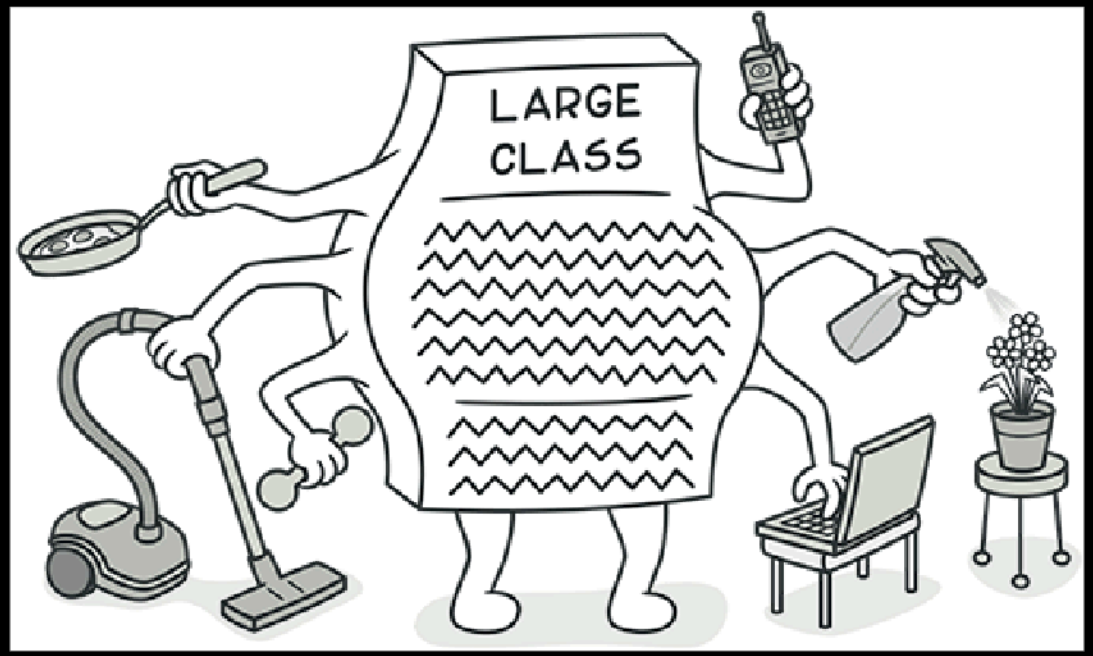
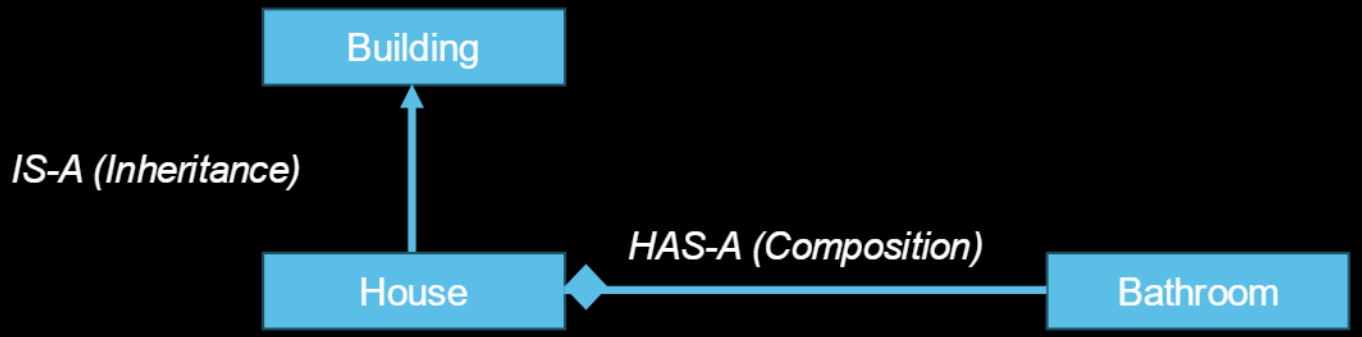


Image from Refactoring.Guru., 2025, https://logicmojo.com/abstraction-in-oops

Best Practices: Favor Composition Over Inheritance

- **"Has-a"** relationships are often better than **"is-a"** relationships.
- Composition leads to more **flexible** and **reusable** code.



Best Practices: Interface Segregation Principle

- Clients should **not** be forced to depend on methods they do not use.
- Create **smaller**, more **specific** interfaces instead of large, general-purpose ones.

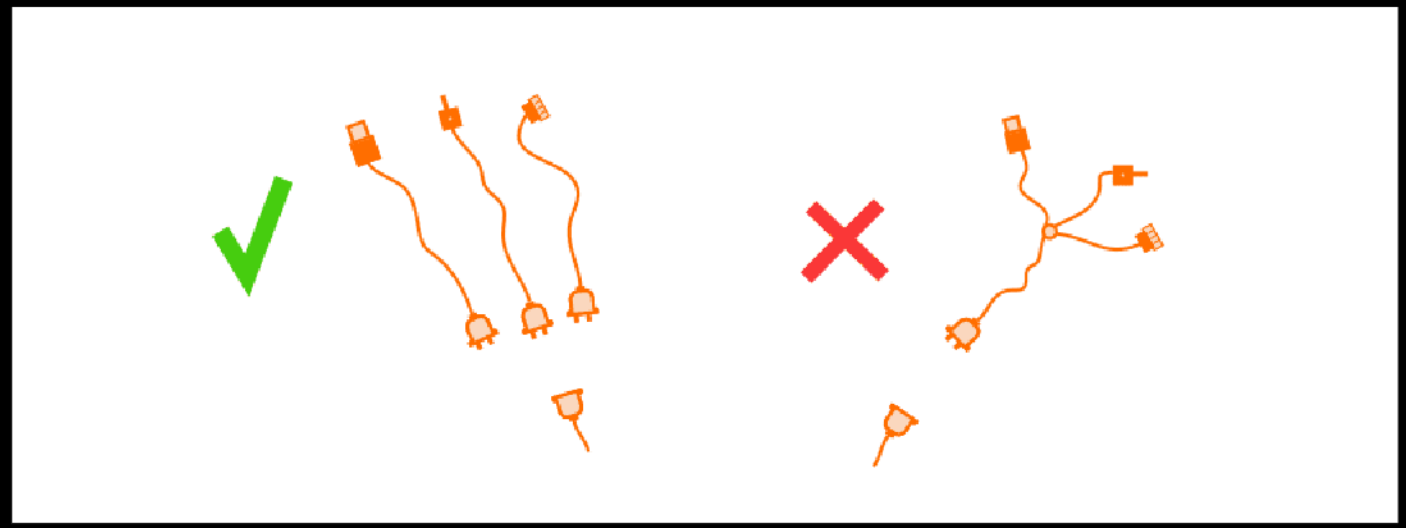


Image from Michał Romańczuk, 15 December 2021, https://accesto.com/blog/solid-php-solid-principles-in-php/

Code Example: ISP Violation & Solution

```python
from abc import ABC, abstractmethod

class Worker(ABC):
    @abstractmethod
    def work(self):
        pass

    @abstractmethod
    def eat(self):
        pass

class NormalWorker(Worker):
    def work(self):
        print("Normal worker is working")

    def eat(self):
        print("Normal worker is eating")

class Robot(Worker):
    def work(self):
        print("Robot is working")

    def eat(self):
        # Robots don't eat, but we have to implement this method
        pass
```

```python
from abc import ABC, abstractmethod

class Workable(ABC):
    @abstractmethod
    def work(self):
        pass

class Eatable(ABC):
    @abstractmethod
    def eat(self):
        pass

class NormalWorker(Workable, Eatable):
    def work(self):
        print("Normal worker is working")

    def eat(self):
        print("Normal worker is eating")

class Robot(Workable):
    def work(self):
        print("Robot is working")
```

Scenario Question

- Question: You are designing a system for managing different types of documents (PDF, Word, Text). How would you use abstraction and interfaces to design this system?
- Think about the key functionalities and how to abstract them.

* Take a minute to think individually, then discuss with a partner, and finally share your thoughts with the class.

Example Solution: Document Management System

- Abstraction: Focus on core document operations like *open*, *save*, *print*, and *convert*.
- Interface: Define a *Document* interface with these operations.
- Concrete Classes: Create classes like *PDFDocument*, *WordDocument*, and *TextDocument*, each implementing the *Document* interface and providing specific implementations for each operation.

```python
from abc import ABC, abstractmethod

class Document(ABC):
    @abstractmethod
    def open(self):
        pass

    @abstractmethod
    def save(self):
        pass

    @abstractmethod
    def print_document(self):
        pass

    @abstractmethod
    def convert(self, format):
        pass
```

```python
class PDFDocument(Document):
    def open(self):
        print("Opening PDF document")
        # PDF-specific opening logic

    def save(self):
        print("Saving PDF document")
        # PDF-specific saving logic

    def print_document(self):
        print("Printing PDF document")
        # PDF-specific printing logic

    def convert(self, format):
        print(f"Converting PDF to {format}")
        # PDF-specific conversion logic

# Similar implementations for WordDocument and TextDocument
```

Key Takeaways

Abstraction

- Simplifies complex systems by focusing on essential characteristics.
- Achieved through classes, abstract classes, and duck typing.
- Reduces complexity, increases reusability, and improves maintainability.

Interfaces

- Define contracts for classes, specifying required methods.
- Implemented using abstract base classes (ABCs) in Python.
- Promote loose coupling, polymorphism, and testability.

Q & A

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Lab